RP: Extrusion Networks to Overcome the Curse of (Simulation) Dimensionality

Harsha Vardhan Dasari[1] **Arya Vaidya**[2] **Shital Adhikari**[1] **Nikhil Muralidhar**[1]

[1]Stevens Institute of Technology [2]Georgia Institute of Technology
{hdasari, sadhikar1, nmurali1}@stevens.edu, avaidya82@gatech.edu

Abstract

Numerical solvers for time-dependent partial differential equations (PDEs) remain prohibitively expensive in high-dimensional, multi-parameter regimes. Neural surrogates such as Fourier Neural Operators (FNO) and UNet-style architectures offer fast approximations, but their accuracy degrades sharply under data-scarcity conditions, and they demand large parameter budgets to generalize across PDE configurations. We introduce *Extrusion Networks*, a lightweight, transfer-learning-guided hypernetwork architecture in which the weights of a dilated convolutional ResNet backbone are *generated* dynamically, conditioned on PDE parameters and timestep. By extruding pretrained one-dimensional filter banks into higher-dimensional kernels through a bounded sinusoidal modulation, a single $\sim$500K-parameter model (combining both 1D and 2D weights) generalizes across PDE configurations without retraining. With filters pretrained on all 14 parameter configurations of the 1D Kuramoto–Sivashinsky system, Extrusion Networks are trained on 2D data from only three configurations and evaluated on all 14, including 11 held out from 2D training. Across these configurations, the model achieves the lowest cumulative rollout MAE among the evaluated models at every reported horizon up to 20 steps. Cumulative rollout MAE is approximately **33–71%** lower than that of the strongest evaluated baseline, a UNet trained with a 5-step rollout loss, across the reported horizons.

Introduction

High-fidelity simulation of PDE systems underpins engineering design, climate modeling, and computational physics, yet the cost of classical solvers grows with spatial resolution, the number of parameter configurations, and rollout horizon – the *curse of (simulation) dimensionality*. Learned surrogates, including Fourier Neural Operators (FNO) (Li et al. 2021) and UNet-based solvers (Gupta and Brandstetter 2022), offer an alternative to repeated numerical simulation. Their practical value, however, depends on learning accurate dynamics from limited simulation data and maintaining accuracy across PDE configurations.

One source of reusable information is the dynamics of related lower-dimensional PDEs, for which training data can be less expensive to generate. PreLowD (Hemmasian and Barati Farimani 2024) investigates this approach by pretraining neural operators in lower dimensions and transferring learned parameters to higher-dimensional problems. Building on this direction, we investigate whether pretrained one-dimensional convolutional filters can serve as reusable components of higher-dimensional kernels generated conditionally on PDE parameters and timestep.

We propose **Extrusion Networks**, a hypernetwork architecture that dynamically generates convolutional kernels for a dilated ResNet backbone. The hypernetwork (Ha, Dai, and Le 2017) is conditioned on PDE parameters, a positional encoding of the timestep, and coordinates identifying the kernel chunk to be generated. Its outputs are combined with pretrained 1D ResNet filter banks through an extrusion operation followed by a sine nonlinearity.

Contributions.

(1) A conditioned weight-extrusion architecture that combines hypernetwork outputs with pretrained 1D filters to generate bounded higher-dimensional convolutional kernels, sharing a single model across PDE configurations and timesteps. (2) An adaptation of RevIN (Kim et al. 2022) in which affine scale and shift parameters are predicted from PDE parameters, incorporating configuration information into normalization. (3) An evaluation on 2D Kuramoto–Sivashinsky showing that Extrusion Networks achieve the lowest cumulative rollout MAE among the evaluated models at every reported horizon up to 20 steps, with approximately 33–71% lower error than the strongest evaluated baseline.

Related Work

Neural PDE surrogates and rollout evaluation. Neural operators learn mappings between function spaces and provide a framework for approximating PDE solution operators (Kovachki et al. 2023). FNO (Li et al. 2021) parameterizes these mappings through Fourier-domain operations, while FFNO (Tran et al. 2023) introduces separable spectral layers and modified residual connections. Convolutional approaches, including UNet- and ResNet-based models, provide alternative architectures for predicting time-dependent PDE dynamics (Gupta and Brandstetter 2022). PDEBench (Takamoto et al. 2022) and APEBench (Koehler et al. 2024) support systematic evaluation of neural PDE models, with APEBench emphasizing autoregressive training and rollout metrics. PDE-Refiner (Lippe et al. 2023) further investigates the difficulty of maintaining accurate predictions over long rollouts.

Pretraining and transfer across spatial dimensions.

Copyright © 2027, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

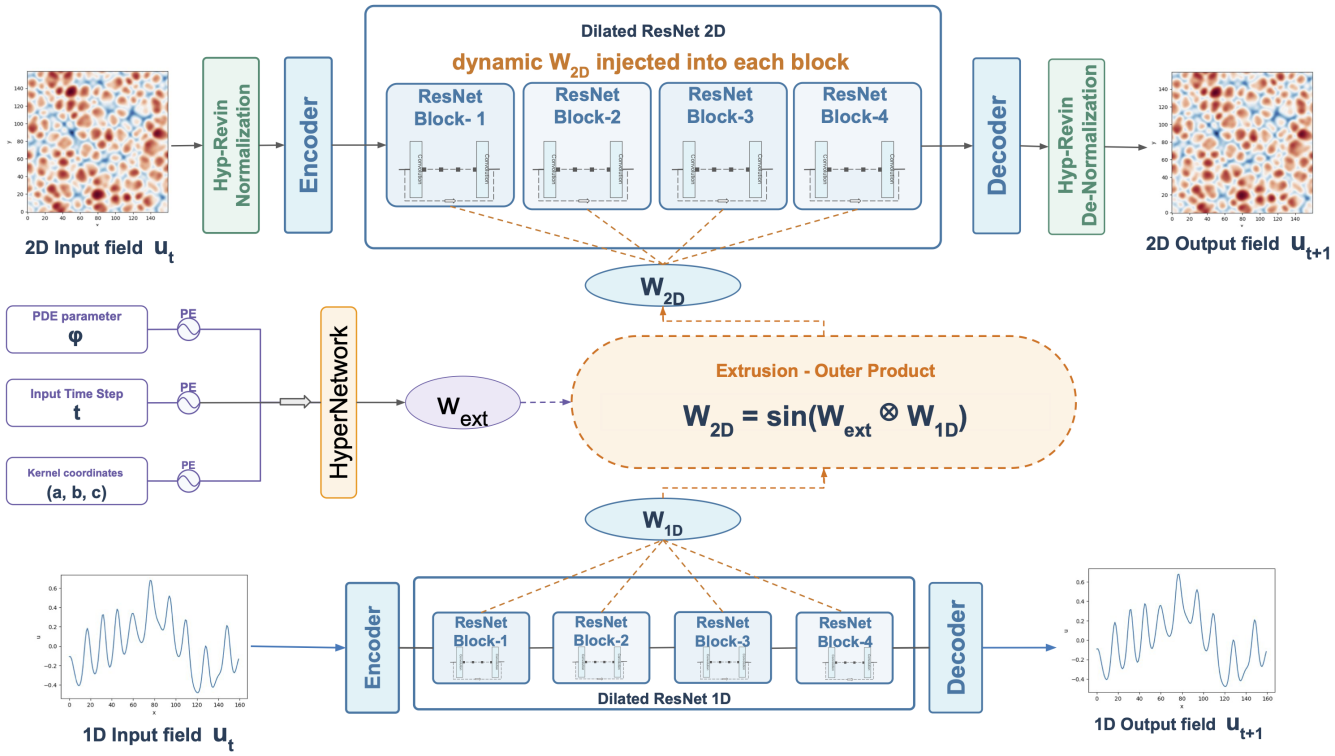


Figure 1: Extrusion Network architecture. A hypernetwork conditioned on PDE parameters, timestep, and kernel-chunk coordinates combines its outputs with pretrained 1D filters through sinusoidal weight extrusion to generate 2D convolutional kernels.

Pretraining offers a way to reuse information across PDE datasets, although its effectiveness depends on the model, training distribution, and downstream task (Zhou et al. 2024). PreLowD (Hemmasian and Barati Farimani 2024) specifically investigates pretraining on lower-dimensional PDEs and transferring learned parameters to higher-dimensional problems using FFNO. The distinction lies in how pretrained information is reused: Extrusion Networks combine pretrained 1D convolutional filters with hypernetwork-generated tensors to construct kernels conditioned on PDE parameters, timestep, and kernel-chunk coordinates.

Hypernetworks and conditional normalization. Hypernetworks generate the weights of a target network using another network (Ha, Dai, and Le 2017). Conditioning can also act on network features: FiLM (Perez et al. 2018) applies affine transformations determined by conditioning information, while RevIN (Kim et al. 2022) uses reversible normalization to address distribution shift in time-series forecasting. Our normalization scheme combines reversible normalization with PDE-conditioned prediction of affine scale and shift.

Method

Problem setup

We learn an autoregressive surrogate $u_{t+1} = f_\theta(u_t; \phi, t)$ for solution fields u_t of a parametric PDE family, where ϕ denotes PDE parameters (e.g., the Kuramoto–Sivashinsky viscosity and domain-length coefficients) and t the timestep. The goal is a single model that remains accurate across the joint (ϕ, t) space and over long rollouts.

Architecture

Figure 1 summarizes the architecture. **Conditioning.** The hypernetwork always emits a fixed-shape $16\times16\times1$ weight chunk; each convolution's full $48\times48\times3$ kernel is assembled from 9 such chunks. A coordinate triple (a, b, c) identifies which chunk is being generated: a indexes the residual block ($a \in \{1, \ldots, 4\}$), b indexes the convolution within that block ($b \in \{1, \ldots, 7\}$), and c indexes the chunk within that convolution's kernel ($c \in \{1, \ldots, 9\}$). For each triple, the PDE parameters ϕ and the sinusoidal positional encoding of the timestep, $\mathrm{PE}(t)$, are concatenated with a sinusoidal encoding of the chunk coordinates, $\mathrm{PE}(a, b, c)$, to form the conditioning vector

$$z_i = \left[\phi;\ \mathrm{PE}(t);\ \mathrm{PE}(a, b, c)\right],$$

which is passed to the hypernetwork. The coordinate encoding tells the hypernetwork which chunk of which convolution it is generating.

PDE-aware normalization(Hyp-Revin). Input fields are normalized with RevIN (Kim et al. 2022), but rather than learned static affine parameters, a small MLP conditioned on the PDE parameters ϕ predicts the scale and shift, so normalization statistics adapt to the PDE configuration. Outputs are denormalized with the inverse transform.

Backbone. An encoder convolution lifts the input to a 48-channel latent representation, followed by 4 resid-

ual blocks (He et al. 2016), each containing 7 dilated convolutions (Yu and Koltun 2016) with dilation rates $(1, 3, 5, 7, 5, 3, 1)$ (28 convolutions in total), with a skip connection around each block, and a decoder head. The symmetric dilation schedule grows the receptive field cheaply, which is critical to capturing the long-range spatial correlations of chaotic dynamics.

Weight extrusion. For each coordinate triple (a, b, c), the conditioning vector z_i is passed through the hypernetwork to produce a $16\times16\times1$ chunk; the 9 chunks of each convolution are assembled into an extrusion tensor W_{ext}. The convolution's effective $48\times48\times3$ kernel is

$$W = \sin(W_{\text{ext}} \otimes W_{\text{1D}})\,. \tag{1}$$

where W_{1D} are pretrained 1D ResNet filters. The Outer product extrudes the 1D filter bank into the full kernel, and $\sin(\cdot)$ keeps the generated weights bounded and smooth, which we find essential for stable long rollouts. Biases decompose as $b = z_b + b_{\text{1D}}$ and are configuration-independent.

Experiments

Dataset Description. All simulation data are generated using APEBench (Koehler et al. 2024) for the Kuramoto–Sivashinsky (KS) system, using its default temporal resolution and frame length throughout. The KS equation is a nonlinear, fourth-order PDE that combines destabilizing second-order dynamics with stabilizing fourth-order dissipation. A standard one-dimensional form is

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + \nu\frac{\partial^2 u}{\partial x^2} + \kappa\frac{\partial^4 u}{\partial x^4} = 0, \tag{2}$$

where $u(x, t)$ denotes the evolving field and $\nu, \kappa > 0$ control the second- and fourth-order terms, respectively. The nonlinear term couples spatial modes, while the second-order term amplifies long-wavelength perturbations and the fourth-order term damps short-wavelength fluctuations. Their interaction can produce complex spatiotemporal dynamics. The coefficients and domain length determine which spatial modes are unstable, making KS a useful setting for studying prediction across different dynamical regimes.

To evaluate transfer across these dynamical regimes, we use 14 KS parameter configurations that vary the viscosity and domain-length coefficients, spanning smooth, near-laminar field dynamics through fully chaotic regimes. The 1D filter bank is pretrained on all 14 configurations, with 500 simulated trajectories per configuration. The 2D Extrusion Network is trained on a deliberately scarce subset of only 3 of these 14 configurations, each contributing 25 trajectories. Evaluation spans all 14 configurations – the 3 seen during 2D training and 11 held out entirely from 2D training – using 20 held-out test trajectories per configuration, which lets us assess both in-distribution accuracy and generalization across the smooth-to-chaotic parameter range. Cumulative rollout MAE is computed by autoregressively unrolling each model from the initial field of a held-out trajectory and comparing against the ground-truth solution for up to 20 steps.

Baseline Methods. We compare Extrusion Networks against Five baselines spanning fixed-weight and transfer-learning approaches. All baselines are reimplemented with approximately 500K parameters to control for parameter count when comparing architectural and training differences.

UNet (Ronneberger, Fischer, and Brox 2015; Takamoto et al. 2022) is a convolutional encoder–decoder with skip connections, trained under a single-step objective. It provides a fixed-weight convolutional baseline for autoregressive prediction.

ResNet (He et al. 2016) is a residual convolutional backbone without hypernetwork-based weight conditioning. It provides a comparison between fixed convolutional weights and the dynamically generated kernels used by Extrusion Networks.

FNO (Li et al. 2021) parameterizes the solution operator through learned Fourier-domain convolutions. It provides a spectral neural-operator baseline alongside the convolutional architectures.

UNet-Multistep uses the same architecture as UNet but is trained with a 5-step rollout loss rather than a single-step objective. Comparing these two models evaluates the effect of multistep training while keeping the architecture fixed.

PreLowD (Hemmasian and Barati Farimani 2024) pre-trains a neural operator on a lower-dimensional PDE and transfers the learned parameters to the higher-dimensional target problem. It provides a transfer-learning baseline for evaluating the reuse of lower-dimensional dynamics.

Training Settings. All models are trained using mean squared error (MSE) loss. Our training configuration uses 500 epochs, a batch size of 48, and the AdamW optimizer with an initial learning rate of 1×10^{-3}. Learning rates are scheduled using cosine annealing with warm restarts (`CosineAnnealingWarmRestarts`, $T_0 = 20$). This configuration is applied to all models except the PreLowD model, which follow their respective training procedures. All models are evaluated using the same set of 10 random seeds.

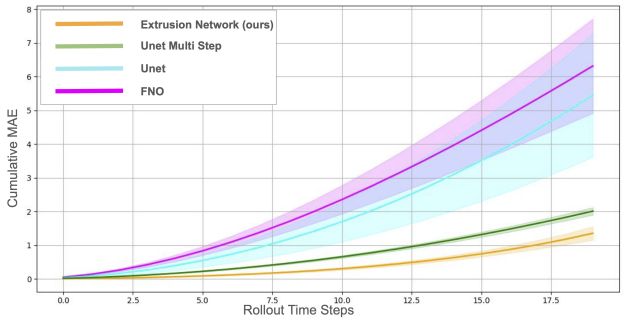


Figure 2: Cumulative rollout MAE (mean ± std, shaded) over 20 rollout steps for the strongest models. The Extrusion Network (orange) accumulates error most slowly and with the tightest variance band, despite using only single-step training.

Accuracy. Table 1 and Figure 2 report cumulative rollout MAE. Extrusion Networks achieve the lowest error at every horizon, reducing cumulative MAE relative to the strongest baseline (UNet-Multistep) by approximately **71%** at 1 step, **63%** at 5 steps, **56%** at 10 steps, **46%** at 15

Table 1: Cumulative rollout MAE (mean $\pm$ std) on 2D Kuramoto–Sivashinsky at increasing rollout horizons. Lower is better; best in bold. "—" indicates the model diverged at that horizon.

Model	1-step	5-step	10-step	15-step	20-step
UNet	0.033 ± 0.011	0.450 ± 0.153	1.541 ± 0.530	3.225 ± 1.103	5.447 ± 1.832
ResNet	0.267 ± 0.158	3.639 ± 2.195	12.636 ± 8.409	26.150 ± 18.727	44.460 ± 33.045
FNO	0.078 ± 0.023	0.809 ± 0.148	2.310 ± 0.412	4.206 ± 0.848	6.316 ± 1.398
UNet-Multistep	0.014 ± 0.001	0.164 ± 0.011	0.550 ± 0.033	1.163 ± 0.066	2.010 ± 0.110
PreLowD (FFNO)	0.453 ± 0.266	5.133 ± 3.443	22.821 ± 17.957	25.410 ± 18.485	—
Extrusion Network (ours)	$\mathbf{0.004 \pm 0.001}$	$\mathbf{0.061 \pm 0.008}$	$\mathbf{0.244 \pm 0.030}$	$\mathbf{0.630 \pm 0.073}$	$\mathbf{1.351 \pm 0.197}$

steps, and **33%** at 20 steps, and by **4–19**$\times$ relative to FNO and the standard UNet. Although the Extrusion Network is trained with a single-step objective only, it outperforms the 5-step-rollout-trained UNet-Multistep at every horizon, including the long-rollout regime that multi-step training targets. PreLowD (FFNO) diverges at the 20-step horizon,, while our model's error grows smoothly, which we attribute to the bounded $\sin$ weight generation stabilizing autoregressive error accumulation.

Generalization. A single trained model transfers across PDE configurations ϕ without retraining; performance on held-out configurations degrades gracefully, and the gap over FNO/UNet widens as training data becomes scarcer, consistent with the transfer-learning inductive bias of the extruded 1D filters.

Conclusion

Extrusion Networks show that a small, transfer-learning-guided hypernetwork that extrudes pretrained 1D filters into configuration-conditioned kernels can outperform fixed-weight neural operators on the chaotic Kuramoto–Sivashinsky system, while remaining data-efficient in training. These results suggest that low-dimensional structure, once learned, can be reused: extruding it into higher dimensions provides a strong inductive bias in the regime where data is scarcest. Future work includes scaling to 3D systems and irregular meshes, extending the extrusion principle to attention-based operators, and coupling the surrogate with downstream design-optimization and uncertainty-quantification loops.

References

Gupta, J. K.; and Brandstetter, J. 2022. Towards Multi-spatiotemporal-scale Generalized PDE Modeling. *arXiv preprint arXiv:2209.15616*.

Ha, D.; Dai, A.; and Le, Q. V. 2017. HyperNetworks. In *International Conference on Learning Representations*.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

Hemmasian, A.; and Barati Farimani, A. 2024. Pretraining a Neural Operator in Lower Dimensions. *Transactions on Machine Learning Research*.

Kim, T.; Kim, J.; Tae, Y.; Park, C.; Choi, J.-H.; and Choo, J. 2022. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. In *International Conference on Learning Representations*.

Koehler, F.; Niedermayr, S.; Westermann, R.; and Thuerey, N. 2024. APEBench: A Benchmark for Autoregressive Neural Emulators of PDEs. In *Advances in Neural Information Processing Systems*.

Kovachki, N.; Li, Z.; Liu, B.; Azizzadenesheli, K.; Bhattacharya, K.; Stuart, A.; and Anandkumar, A. 2023. Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs. *Journal of Machine Learning Research*, 24(89): 1–97.

Li, Z.; Kovachki, N.; Azizzadenesheli, K.; Liu, B.; Bhattacharya, K.; Stuart, A.; and Anandkumar, A. 2021. Fourier Neural Operator for Parametric Partial Differential Equations. In *International Conference on Learning Representations*.

Lippe, P.; Veeling, B. S.; Perdikaris, P.; Turner, R. E.; and Brandstetter, J. 2023. PDE-Refiner: Achieving Accurate Long Rollouts with Neural PDE Solvers. In *Advances in Neural Information Processing Systems*.

Perez, E.; Strub, F.; de Vries, H.; Dumoulin, V.; and Courville, A. 2018. FiLM: Visual Reasoning with a General Conditioning Layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Ronneberger, O.; Fischer, P.; and Brox, T. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. In *Medical Image Computing and Computer-Assisted Intervention*.

Takamoto, M.; Praditia, T.; Leiteritz, R.; MacKinlay, D.; Alesiani, F.; Pflüger, D.; and Niepert, M. 2022. PDEBench: An Extensive Benchmark for Scientific Machine Learning. In *Advances in Neural Information Processing Systems*.

Tran, A.; Mathews, A.; Xie, L.; and Ong, C. S. 2023. Factorized Fourier Neural Operators. In *International Conference on Learning Representations*.

Yu, F.; and Koltun, V. 2016. Multi-Scale Context Aggregation by Dilated Convolutions. In *International Conference on Learning Representations*.

Zhou, A.; Lorsung, C.; Hemmasian, A.; and Barati Farimani, A. 2024. Strategies for Pretraining Neural Operators. *Transactions on Machine Learning Research*.